

Modul 03: Templates

<i>Ziel:</i>	<i>Services verwenden</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~30min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>05.08.2024</i>

1 Liste mit *ngFor darstellen

Um endlich alle Elemente des Arrays darzustellen, verwenden wir die NgFor Directive aus dem Modul @angular/common.

```
<ul>
  <li *ngFor="let task of taskList; let i = index" title="#{{ i }}">
    <app-task-item [item]="task" />
  </li>
</ul>
```

Seit Angular 17 können wir auch die neue Control-Flow Syntax verwenden, welche in ähnlicher Form bereits von vielen anderen Web Frameworks verwendet wird. Sie erfüllt exakt denselben Zweck, aber ist besser lesbar.

```
<ul>
  @for (task of taskList; track $index) {
    <li title="#{{ $index }}"><app-task-item [item]="task" /></li>
  }
</ul>
```

2 Statusleiste stylen

Wir wollen noch die Statusleiste mit folgender Klasse stylen und fügen sie in task-list.component.css ein.

```
.status {
  background-color: aliceblue;
  border: 1px solid gainsboro;
  border-radius: .6em;
}
```

```
.status legend {
  padding: 0 .2rem;
  background-color: inherit;
  border-right: inherit;
  border-bottom: inherit;
  border-radius: inherit;
}

.status p {
  text-align: center;
}
```

Jedoch entsteht der unschöne Effekt, dass die Statusleiste immer angezeigt wird, auch wenn kein Text gesetzt wurde. Dies können wir mit der NgIf Directive aus dem Modul @angular/common beheben.

```
<fieldset class="status" *ngIf="status !== ''">
  <legend> Information </legend>
  <p>{{ status }}</p>
</fieldset>
```

Ebenfalls können wir seit Angular 17 die besser lesbare Control-Flow Syntax verwenden, um unsere Attribut-Liste der HTML-Elemente sauber zu halten.

```
@if (status) {
  <fieldset class="status">
    <legend> Information </legend>
    <p>{{ status }}</p>
  </fieldset>
}
```

3 Texteingabe übernehmen

Um endlich die richtige Texteingabe für die neue Aufgabe anzuzeigen, verwenden wir eine sogenannte Template Reference Variable. Ferner setzen wir eine Referenz auf das Formular, um die Validität des required-Attributes abzufragen.

```
<form #formRef class="grid">
  <label for="title">Aufgabe</label>
  <input id="title" #titleRef type="text" required
    (input.enter)="addTask($event, titleRef.value)" />
  <br />
  <button (click)="addTask($event, titleRef.value)">Neue Aufgabe</button>
</form>
```

4 Texteingabe validieren

Im Code Behind können wir die Template Reference Variable mit dem ViewChild-Decorator abfragen.

```
@ViewChild('formRef') formRef!: ElementRef<HTMLFormElement>;

get valid() {
  return this.formRef?.nativeElement.checkValidity();
}

// ...

addTask(e: Event, title: string) {
  e.preventDefault();

  if (!this.valid) {
    this.setStatus('Bitte geben Sie den Titel der Aufgabe ein.');
```

```
  } else {
    // ...
```

```
    this.formRef.nativeElement.reset();
  }
}
```

5 Statuselement als eigene Komponente

Um das Statuselement wiederverwendbar zu machen, erstellen wir eine eigene Komponente namens shared/status-element. Dorthin verschieben wir die Styles und das HTML. Hierbei verwenden wir Content Projection um den Inhalt darzustellen.

```
<fieldset class="status">
  <legend>
    <ng-content select="[title]"></ng-content>
  </legend>
  <p>
    <ng-content></ng-content>
  </p>
</fieldset>
```

In unserer task-list können wir das neue Element so einbinden.

```
@if (status) {
  <app-status-element>
    <span title>Information</span>
    {{ status }}
  </app-status-element>
}
```